```
expect(client).to receive(:current_price).at_least(:once)
expect(client).to receive(:current_price).at_most(:thrice)
```

If your code calls the method too many or too few times, you'll get an unsatisfied expectation:

```
>> expect(client).to receive(:current_price).at_most(:twice) \
>>   .and_return(130.0)
=> #<RSpec::Mocks::VerifyingMessageExpectation                              ↩
#<InstanceDouble(NasdaqClient) (anonymous)>.current_price(any arguments)>
>> stock_ticker = StockTicker.new(client)
=> #<StockTicker>
>> stock_ticker.price('AAPL')
=> 130.0
>> stock_ticker.price('AAPL')
=> 130.0
>> stock_ticker.price('AAPL')
RSpec::Mocks::MockExpectationError: (InstanceDouble(NasdaqClient)           ↩
(anonymous)).current_price("AAPL")
    expected: at most 2 times with any arguments
    received: 3 times with arguments: ("AAPL")
    « backtrace truncated »
```

It's worth noting that you can't combine the at_least and at_most constraints.

Ordering

Normally, RSpec doesn't care what order you send messages to a test double:

14-customizing-test-doubles/02/setting_constraints.rb

```
expect(greeter).to receive(:hello)
expect(greeter).to receive(:goodbye)

# The following will pass:
greeter.goodbye
greeter.hello
```

If you do need to enforce a specific order, add the ordered modifier:

14-customizing-test-doubles/02/setting_constraints.rb

```
expect(greeter).to receive(:hello).ordered
expect(greeter).to receive(:goodbye).ordered

# The following will fail:
greeter.goodbye
greeter.hello
```

Using ordered is a sign that your specs may be too coupled to one particular implementation. In the next chapter, we'll talk more about code smells like this one, and how to deal with them.